



SMART TRAFFIC

Extensión a gestión inteligente de residuos urbanos

<https://github.com/mrcl29/smart-bins-iot>



07 DE MAYO DE 2026

TONI ROIG MUÑOZ

DAVID SANCHEZ LAGUNA

MARC LLOBERA VILLALONGA

JOSÉ JAVIER DÍAZ LEÓN

Tabla de Contenidos

1. Introducción.....	1
2. Arquitectura del sistema.....	2
2.1. Dispositivos del sistema.....	2
2.2. Flujo de comunicación.....	3
3. Diseño de la comunicación MQTT.....	4
3.1. Topics utilizados.....	4
3.2. Estructura de mensajes JSON.....	4
3.3. AWS IoT.....	8
4. Implementación realizada.....	10
4.1. Abstracción de la capa de comunicación.....	11
4.2. Modelado de dispositivos y entidades.....	11
4.3. Lógica de negocio y orquestación.....	11
4.4. Interacción entre Componentes.....	12
5. Simulación y ejecución.....	13
6. Resultados obtenidos.....	14
7. Conclusiones.....	15

1.Introducción

El presente proyecto consiste en la ampliación del sistema Smart Traffic proporcionado inicialmente en la asignatura, incorporando funcionalidades relacionadas con la gestión inteligente de residuos urbanos mediante dispositivos IoT y comunicaciones MQTT.

El sistema original estaba centrado en la monitorización y simulación de tráfico inteligente, utilizando vehículos, carreteras y eventos distribuidos para representar el comportamiento de una infraestructura urbana conectada. A partir de esta base, se ha desarrollado una extensión orientada a la integración de contenedores inteligentes y camiones de basura capaces de comunicarse con el sistema de tráfico ya existente.

El objetivo principal de esta ampliación es permitir que los contenedores inteligentes informen de su estado de llenado y ubicación en tiempo real, mientras que los camiones de recogida puedan utilizar dicha información junto con el estado del tráfico para optimizar sus rutas y mejorar la eficiencia del servicio.

Toda la comunicación entre dispositivos se ha realizado utilizando MQTT, siguiendo una arquitectura basada en publicación/suscripción. Además, se han definido estructuras JSON estandarizadas para los mensajes intercambiados entre los distintos dispositivos del sistema.

2.Arquitectura del sistema

La arquitectura implementada parte del sistema Smart Traffic original y añade nuevos dispositivos y canales de comunicación relacionados con la gestión de residuos urbanos.

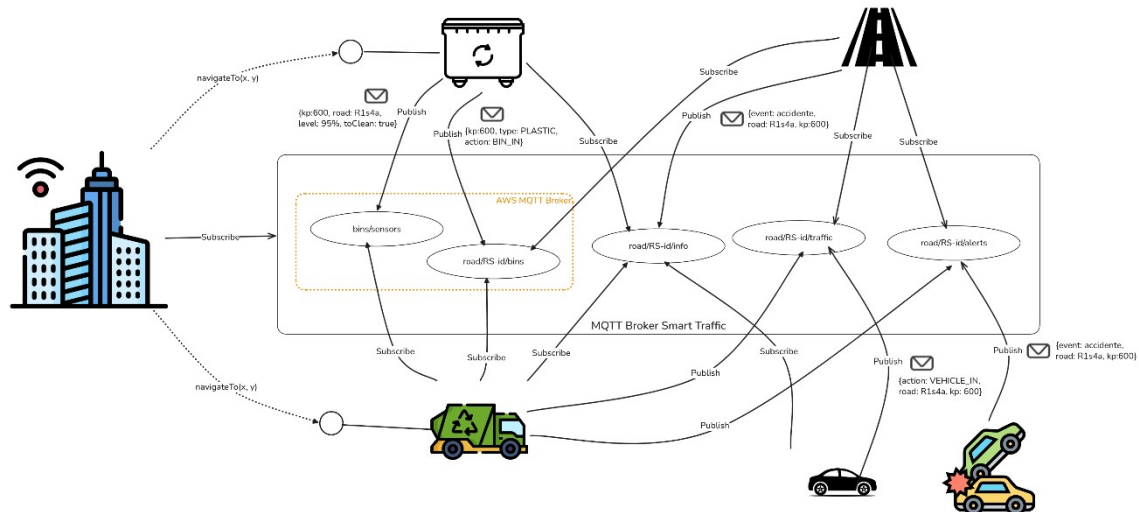


Ilustración 1: Diagrama de Comunicaciones

La imagen anterior representa el flujo completo de comunicación entre todos los dispositivos y servicios del sistema.

2.1. Dispositivos del sistema

El sistema está compuesto por cuatro tipos principales de dispositivos:

- **Vehículos:** los vehículos representan los dispositivos móviles del sistema Smart Traffic. Estos publican eventos relacionados con el tráfico y alertas de accidentes en diferentes topics MQTT. Además, pueden suscribirse a información generada por las carreteras para conocer el estado del tráfico en tiempo real.
- **Carreteras:** las carreteras actúan como nodos de agregación de información. Estas reciben eventos de tráfico, alertas y cambios de posición de los contenedores inteligentes. Posteriormente, procesan toda esta información y generan estados globales del sistema.
- **Contenedores:** los contenedores inteligentes son dispositivos IoT capaces de monitorizar su nivel de llenado y comunicarlo al sistema. También pueden informar de su localización actual dentro de un segmento de carretera.
- **Camiones de basura:** los camiones de basura constituyen una extensión del sistema original de vehículos. Estos dispositivos utilizan la infraestructura Smart Traffic existente para desplazarse por la ciudad y, además, reciben información de los contenedores inteligentes con el objetivo de optimizar las rutas de recogida.

2.2. Flujo de comunicación

La comunicación del sistema sigue un modelo basado en publicación y suscripción mediante MQTT.

Los contenedores inteligentes publican periódicamente información sobre su nivel de llenado utilizando el topic *bins/sensors*. Los camiones de basura se encuentran suscritos a dicho canal para detectar qué contenedores requieren recogida.

Por otra parte, los vehículos y camiones publican eventos de tráfico y movimientos utilizando los topics asociados a cada segmento de carretera. Esta información es utilizada por las carreteras para calcular el estado del tráfico y generar información agregada.

Finalmente, las carreteras publican información global del estado de la vía utilizando topics específicos que pueden ser consumidos por otros dispositivos del sistema.

3. Diseño de la comunicación MQTT

Uno de los aspectos más importantes del proyecto ha sido el diseño de los topics MQTT y de las estructuras JSON utilizadas para intercambiar información entre dispositivos.

3.1. Topics utilizados

La siguiente tabla resume los principales topics utilizados en el sistema:

Topic	Publisher	Subscriber	Descripción
road/RS-id/traffic	Vehículos/ Camiones	Carreteras	Eventos de tráfico y movimiento
road/RS-id/alerts	Vehículos	Carreteras	Accidentes y alertas
road/RS-id/info	Carreteras	Vehículos/ Contenedores	Información global del estado de la carretera
road/RS-id/bins	Contenedores	Carreteras/Camiones	Posición de los contenedores
bins/sensors	Contenedores	Camiones	Nivel de llenado y estado

La utilización de topics diferenciados permite desacoplar completamente la comunicación entre dispositivos y facilita la escalabilidad del sistema.

```
protected void publishTrafficEvent(String action, String segmentId, double kp) {  
    String jsonPayload = buildPayload(type: "TRAFFIC", msgPayload);  
    String topic = TOPIC_BASE + "/road/" + segmentId + "/traffic";
```

3.2. Estructura de mensajes JSON

En lo que respecta a los tipos de mensajes JSON que se pueden encontrar, se disponen de los siguientes ejemplos:

<topic-base>/road/{id-segmento}/alerts

```
{
  "id": "MSG_1477472671831",
  "type": "ROAD_INCIDENT",
  "timestamp": 1477472671831,
  "msg": {
    "rt": "traffic::alert",
    "incident-type": "TRAFFIC_ACCIDENT",
    "id": "7129632011",
    "road": "R2",
    "road-segment": "R2S1",
    "starting-position": 10,
    "ending-position": 10,
    "description": "Vehicle Crash",
    "status": "Active",
    "link": "/incident/7129632011"
  }
}
```

Este mensaje es publicado en el topic *es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/{id-segmento}/alerts* a través del broker *tcp://tambori.dsic.upv.es:10083* y se envía por un vehículo cuando detecta un accidente en su segmento de carretera.

La carretera (R2S1) está suscrita a este *topic*, recibe el aviso, actualiza su estado interno y lo redistribuye por *road/{id-segmento}/info*, de modo que todos los demás vehículos y contenedores suscritos a ese *topic* se enteran del accidente en tiempo real.

<topic-base>/road/{id-segmento}/traffic

```
{
  "id": "MSG_1477473530870",
  "type": "TRAFFIC",
  "timestamp": 1477473530870,
  "msg": {
    "action": "VEHICLE_IN",
    "road": "R1",
    "road-segment": "R1S4a",
    "vehicle-id": "SC1477473368403736",
    "position": 520,
    "role": "MedicalAssistance"
  }
}
```

El mensaje es publicado en el topic *es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/{id-segmento}/traffic* (nuevamente a través del broker *tcp://tambori.dsic.upv.es:10083*, y el envío se realiza por un vehículo o camión de basura en el momento en que entra en un segmento de carretera.

La carretera (R1S4a) que está suscrita a este *topic*, recibe el evento de entrada, actualiza su recuento de vehículos activos del segmento y redistribuye el estado por *road/{id-segmento}/info*. Cuando el vehículo abandona el segmento, publicará el mensaje complementario con “*action*”: “*VEHICLE_OUT*” (también está la acción *CHECK_IN* a la hora de actualizar su posición en el segmento de carretera).

<topic-base>/road/{id-segmento}/info

```
{
  "id": "MSG_1477474100254",
  "type": "ROAD_STATUS",
  "timestamp": 1477474100254,
  "msg": {
    "rt": "road-segment",
    "road": "R11",
    "road-segment": "R11S3",
    "start-kp": 0,
    "end-kp": 377,
    "length": 377,
    "capacity": 8,
    "num-vehicles": 2,
    "density": 25,
    "status": "Mostly_Free_Flow",
    "max-speed": 30,
    "current-max-speed": 30,
    "code": "1",
    "link": "/segment/R11S3"
  }
}
```

```
{
  "id": "MSG_1477472669651",
  "type": "ROAD_INCIDENT",
  "timestamp": 1477472669651,
  "msg": {
    "rt": "traffic::incident",
    "incident-type": "INCIDENT",
    "id": "712963150",
    "road": "R8",
    "road-segment": "R8S1",
    "starting-position": 584,
    "ending-position": 684,
    "description": "Working Area",
    "status": "Active",
    "link": "/incident/712963150"
  }
}
```

En cuanto al tipo de mensajes mostrados anteriormente, el *topic* en cuestión será en este caso *es/upv/pros/tatami/smartcities/traffic/PTPaterna/road/{id-segmento}/info*. Ambos mensajes JSON (enviados por las carreteras) muestran, por un lado, el estado actual de la carretera (capacidad, velocidad máxima, etc.) y la retransmisión de un incidente activo en el segmento a los suscriptores de *info*.

<topic-base>/road/{id-segmento}/bins

```
{
  "id": "MSG_1477472669651",
  "type": "BIN_POSITION",
  "timestamp": 1477472669651,
  "msg": {
    "deviceId": "BIN_4321412",
    "road": "R11",
    "road-segment": "R11S3",
    "kp": 500,
    "action": "BIN_OUT", // BIN_OUT or BIN_IN
    "type": "PLASTIC",
  }
}
```

El mensaje es publicado en `es/upv/aws/road/{id-segmento}/bins` a través del broker MQTT de AWS IoT, y se envía por los contenedores de basura cuando entran o salen de un segmento de carretera.

A diferencia de los topics anteriores, este circula por el broker de AWS IoT. Lo que implicaría una infraestructura desacoplada de contenedores y camiones de basura en la nube, separada del resto del tráfico.

La carretera y los camiones de basura están suscritos a este topic. La carretera lo usa para mantener su registro de contenedores activos en el segmento y redistribuirlo por */info*. Los camiones lo usan para conocer tanto la posición y el tipo de cada contenedor y establecer así la ruta de recogida más eficiente.

<topic-base>/bins/sensors

```
{
  "id": "MSG_1477472669651",
  "type": "BIN_SENSOR",
  "timestamp": 1477472669651,
  "msg": {
    "deviceId": "BIN_34242",
    "road": "R11",
    "road-segment": "R11S3",
    "kp": 500,
    "level": 95, // 95%
    "toClean": true,
    "type": "PLASTIC",
  }
}
```

Se publica en este caso el mensaje en `es/upv/aws/bins/sensors` nuevamente con el broker MQTT de AWS IoT. Se envía por los contenedores de basura para repostar su estado interno y nivel de llenado. Con la combinación de `bins/sensors` (saber qué contenedores necesitan recogida y de qué tipo) y `road/{id-segmento}/bins` (saber dónde están los contenedores), pueden construir una ruta óptima priorizando los contenedores más llenos y agrupándolos por tipo de residuo y proximidad geográfica.

3.3. AWS IoT

Los topics que no forman parte actualmente del SmartTraffic se han implementado en AWS para mostrar como sería la configuración y diseño de un bróker MQTT en el cloud.

Los topics que usan AWS IoT son:

<topic-base>/bins/sensors y <topic-base>/road/{id-segmento}/bins

Para facilidad de implementación y ejecución en el código se ha usado un solo objeto con una política que tiene todos los privilegios

Tanto los camiones de basura como los contenedores de basura tienen sus propios certificados todos apuuntando al mismo objeto llamado **Sensor_Basuras_Valencia**. Este objeto tiene dos certificados, uno para cada tipo de dispositivo con sus propias políticas permisivas.

Política Camiones de Basura:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowConnectOnlyAsRegisteredThing",
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:${aws:Region}:${aws:AccountId}:client/${iot:Connection.Thing.ThingName}"
    },
    {
      "Sid": "AllowSubscriptionToBinTopics",
      "Effect": "Allow",
      "Action": "iot:Subscribe",
      "Resource": [
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topicfilter/bins/sensors",
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topicfilter/road/*/bins"
      ]
    },
    {
      "Sid": "AllowReceivingBinMessages",
      "Effect": "Allow",
      "Action": "iot:Receive",
      "Resource": [
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topic/bins/sensors",
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topic/road/*/bins"
      ]
    }
  ]
}
```

Política Contenedores de Basura:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowConnectOnlyAsRegisteredThing",
      "Effect": "Allow",
      "Action": "iot:Connect",
      "Resource": "arn:aws:iot:${aws:Region}:${aws:AccountId}:client/${iot:Connection.Thing.ThingName}"
    },
    {
      "Sid": "AllowPublishingToBinTopics",
      "Effect": "Allow",
      "Action": "iot:Publish",
      "Resource": [
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topic/bins/sensors",
        "arn:aws:iot:${aws:Region}:${aws:AccountId}:topic/road/*/bins"
      ]
    }
  ]
}
```

Usamos `${iot:Connection.Thing.ThingName}` en el recurso de `iot:Connect` para que el dispositivo que se conecte lo haga usando un `clientId` MQTT que coincida con el nombre de registro en AWS IoT, evitando la suplantación usando un mismo certificado.

4.Implementación realizada

A continuación se mostrará de forma gráfica, un diagrama que permite ilustrar de manera visual el diseño para cada componente del sistema:

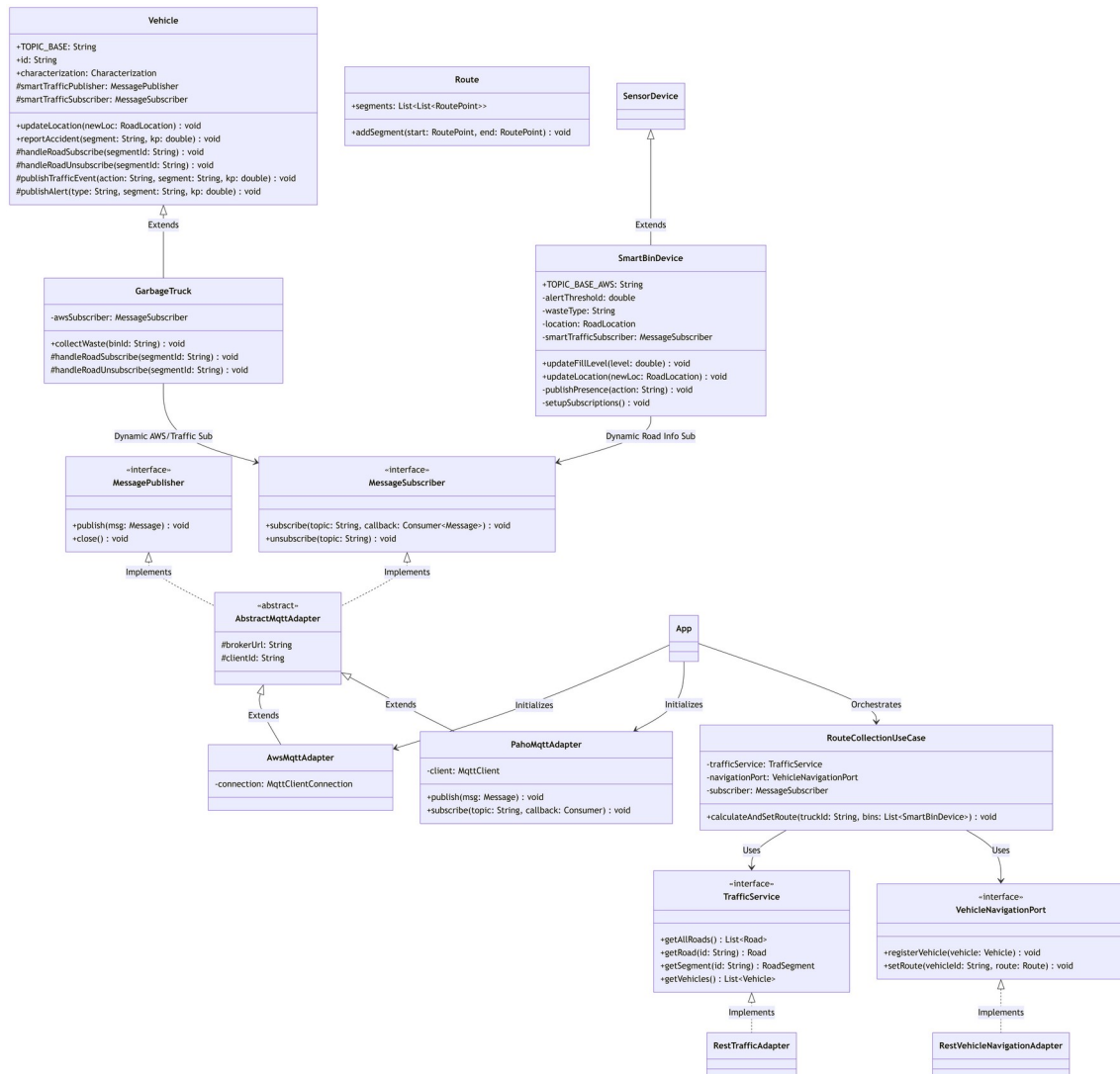


Ilustración 2: Diagrama del código implementado

La arquitectura del sistema se ha diseñado siguiendo los principios de Arquitectura Hexagonal (Puertos y Adaptadores) y Diseño Orientado al Dominio (DDD). Esta elección garantiza que la lógica de negocio del sistema IoT sea independiente de los protocolos de comunicación específicos, facilitando la escalabilidad y el mantenimiento.

4.1. Abstracción de la capa de comunicación

Para gestionar la heterogeneidad de protocolos en entornos IoT, se han definido interfaces de comunicación (puertos) que desacoplan los dispositivos del broker de mensajería:

- **Interfaces de Mensajería:** Se han implementado las interfaces *MessagePublisher* y *MessageSubscriber*. Esto permite que componentes como los vehículos o sensores no dependan de una librería específica (como Paho o AWS SDK), sino de un contrato de comportamiento.
- **Adaptadores MQTT:** Se ha implementado una jerarquía de adaptadores partiendo de una clase abstracta *AbstractMqttAdapter*. *PahoMqttAdapter* se utiliza para la comunicación estándar de tráfico en tiempo real. *AwsMqttAdapter* se especializa en la integración con **AWS IoT Core**, utilizado específicamente por el *GarbageTruck* y *SmartBinDevice* para la gestión de datos persistentes y telemetría avanzada.

4.2. Modelado de dispositivos y entidades

El dominio se divide en entidades físicas que interactúan con la infraestructura urbana:

- **Clase Vehicle y Especializaciones:** La clase *Vehicle* actúa como la base para cualquier unidad móvil, gestionando la ubicación (*updateLocation*) y el reporte de incidentes. El *GarbageTruck* extiende esta funcionalidad, integrando un suscriptor específico de AWS para recibir instrucciones de recolección de residuos.
- **SmartBinDevice (Sensorización):** Representa los contenedores inteligentes. Implementa lógica de umbrales (*alertThreshold*) para notificar su nivel de llenado. Una característica clave es su capacidad de suscripción dinámica basada en la ubicación vial para interactuar con el tráfico circundante.

4.3. Lógica de negocio y orquestación

La inteligencia del sistema reside en la capa de aplicación, destacando el componente **RouteCollectionUseCase**, el cual actúa como el orquestador principal. Utiliza el puerto *TrafficService* para obtener el estado de las carreteras y el *VehicleNavigationPort* para asignar rutas a los camiones. Su función principal es calcular la ruta óptima (*calculateAndSetRoute*) basándose en el estado de llenado de los *SmartBinDevice*.

4.4. Interacción entre Componentes

La implementación fomenta un modelo de **comunicación dirigida por eventos**. Cuando un *SmartBinDevice* supera su umbral de llenado, publica un evento que es procesado por el caso de uso de rutas, el cual, a través de los adaptadores REST (*RestTrafficAdapter*), recalcula la trayectoria del *GarbageTruck* y actualiza su navegación en tiempo real.

5. Simulación y ejecución

Podemos correr el archivo *App.java* para ejecutar una simulación que muestra como sería la interacción de los distintos dispositivos usando la extensión de las SmartBins. En esta hay varios dispositivos interactuando normalmente hasta que unos contenedores de basura avisan que estan llenos, seguidamente se anuncia un accidente en la carretera de los contenedores por lo que estos se mueven a una carretera despejada para que los camiones de basura puedan realizar la ruta cómodamente.



6.Resultados obtenidos

Tras la ampliación del sistema Smart Traffic, se consiguió integrar correctamente la gestión inteligente de residuos dentro de la arquitectura original.

Los principales resultados obtenidos fueron:

- Integración de contenedores inteligentes dentro del sistema IoT.
- Comunicación distribuida mediante MQTT.
- Implementación de mensajes JSON estructurados.
- Reutilización de la infraestructura Smart Traffic original.
- Simulación funcional de rutas de recogida de residuos.
- Comunicación entre camiones, carreteras y contenedores.

7. Conclusiones

La ampliación desarrollada demuestra cómo un sistema Smart Traffic puede extenderse para incorporar nuevos servicios urbanos inteligentes reutilizando la infraestructura existente.

La utilización de MQTT y de una arquitectura desacoplada basada en publicación/suscripción permitió integrar fácilmente nuevos dispositivos y funcionalidades sin necesidad de modificar significativamente el núcleo del sistema original.

Además, el uso de mensajes JSON estructurados y componentes reutilizables mejoró la mantenibilidad y escalabilidad del proyecto.

Finalmente, el proyecto permitió simular un entorno IoT distribuido donde vehículos, carreteras y contenedores inteligentes colaboran para optimizar la gestión.